

Zone Généraliste S2 – ZG2
« Dimensionnement d'un train d'atterrissage d'avion ».
Printemps 2023

Présentation ARDUINO – Semaine 2 - « Capteur Lidar – Carte arduino »

1 – Qu'est-ce que c'est ?



Une communauté



Une carte à micro-contrôleur



Un outil de développement

Les avantages : Pas cher, Environnement de programmation clair et simple, multiplateforme (windows, linux), logiciel et matériel open source.

Une carte Arduino est une petite (5,33x6,85 cm) carte électronique équipée d'un micro-contrôleur. Le micro-contrôleur permet, à partir d'événements détectés par des capteurs, de programmer et commander des actionneurs ; la carte Arduino est une interface programmable.



La carte s'interface au PC par l'intermédiaire de sa prise USB. La carte s'alimente par la prise jack d'alimentation (utilisation autonome) mais peut être alimentée par l'USB (en phase de développement).

2 – Développement d'un projet

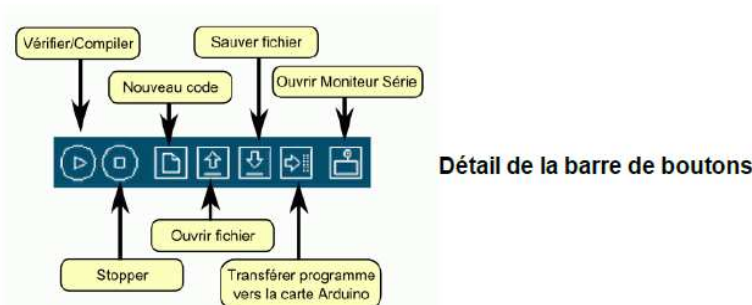
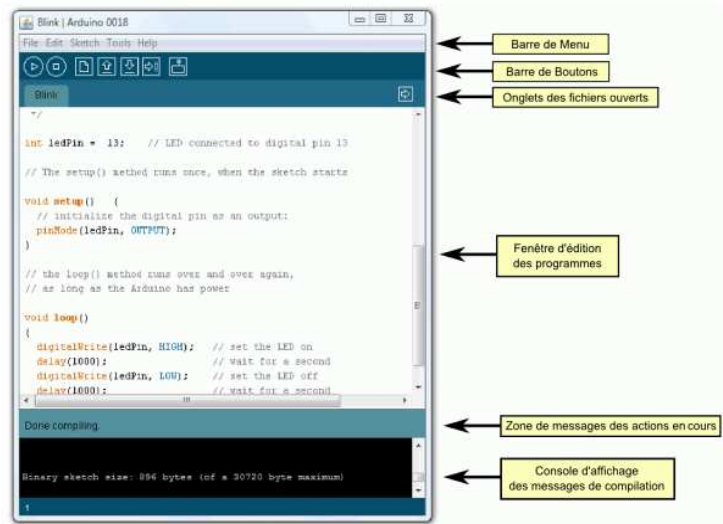
Le développement sur Arduino est très simple :

- On code l'application : le langage Arduino est basé sur le langage C.
- Une fois que le code est développé et compilé, il suffit de « téléverser » le code dans la carte cible à partir de la liaison USB. Il est possible de retirer le câble USB. La carte Arduino est autonome, elle nécessite alors une alimentation électrique (pile 9V).

```
void setup() {
  pinMode(2, OUTPUT);
}

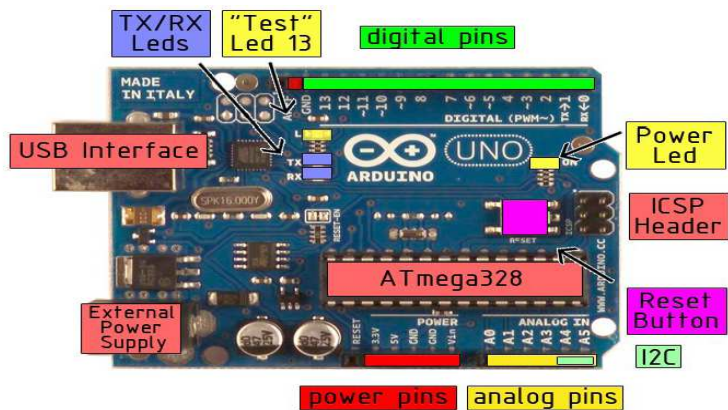
void loop() {
  digitalWrite(2, HIGH);
  delay(1000);
  digitalWrite(2, LOW);
  delay(1000);
}
```





3 – Gros plan sur la carte

Microcontroller	ATmega328
Operating Voltage	5V
Input Voltage (recommended)	7-12V
Input Voltage (limits)	6-20V
Digital I/O Pins	14 (of which 6 provide PWM output)
Analog Input Pins	6
DC Current per I/O Pin	40 mA
DC Current for 3.3V Pin	50 mA
Flash Memory	32 KB of which 0.5 KB used by bootloader
SRAM	2 KB
EEPROM	1 KB
Clock Speed	16 MHz



External power supply : Permet de brancher une alimentation (pile, batterie, adaptateur secteur, Vin de 7 à 12V).

ATmega 328 : Stocke le programme et l'exécute.

Analog pins : Permet de brancher des entrées analogiques.

Digital pins : Permet de brancher des entrées ou sorties logiques.

USB Interface : Permet d'alimenter la carte en énergie électrique (5V) et de « téléverser » le programme dans la carte cible.

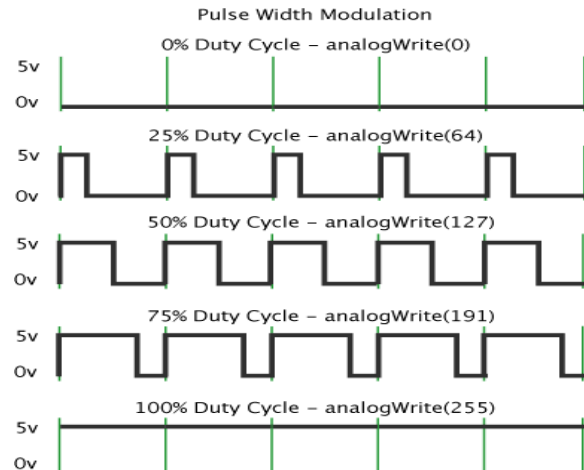
3. 1– Les entrées/sorties numériques : D0 à D13

Chacune des broches D0 à D13 peut être configurée par programmation en entrée ou en sortie. Il est donc possible d'avoir par exemple les broches 2 et 3 configurées comme des entrées (interrupteurs, bouton poussoir) et les broches 7, 8 et 9 configurées comme des sorties (Led, ...).

Les signaux véhiculés par ces broches sont des signaux logiques, c'est-à-dire qu'ils ne peuvent prendre que 2 états : Haut (5V) et Bas (0V), par rapport à la broche masse, qui elle est toujours à 0 V.

Attention : Les broches ne peuvent pas fournir un courant supérieur à 40mA.

A noter que les broches 3, 5, 6, 9, 10 et 11 sont des broches permettant la PWM (Pulse Width Modulation). La PWM est un signal de fréquence fixe et de rapport cyclique variable.



Ces entrées/sorties sont qualifiées de numériques, de digitales ou logiques.

3.2 – Les entrées analogiques : A0 à A5

Contrairement aux entrées/sorties numériques qui ne peuvent prendre que 2 états HAUT et BAS, ces 6 entrées peuvent prendre admettre un millier de valeurs analogiques comprises entre 0 et 5V.

Le logiciel comprend aussi un moniteur série (liaison avec l'écran de l'outil de développement) qui permet d'afficher des messages émis par la carte Arduino et d'envoyer des caractères vers la carte Arduino.

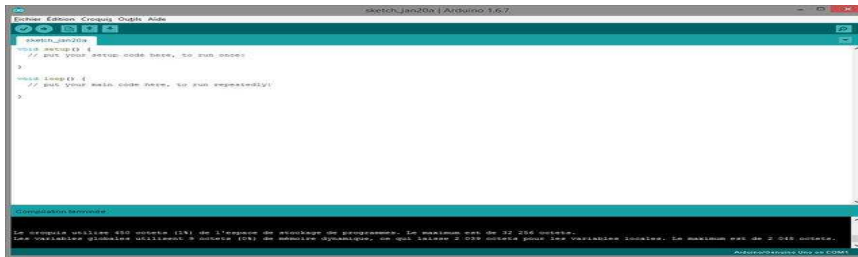
Le site Arduino : <http://www.arduino.cc>. Traduit en français (partiellement) : <http://www.arduino.cc/fr>

4 – Prise en main de la carte Arduino.

A l'ouverture d'un projet, on se retrouve avec les lignes suivantes :

```
1 void setup() {  
2   // put your setup code here, to run once:  
3  
4 }  
5  
6 void loop() {  
7   // put your main code here, to run repeatedly:  
8  
9 }
```

Tout qui se trouve à droite // concerne des commentaires. Ces commentaires sont ignorés lors de la compilation.



Un programme Arduino est toujours composé de deux fonctions : setup() et loop().

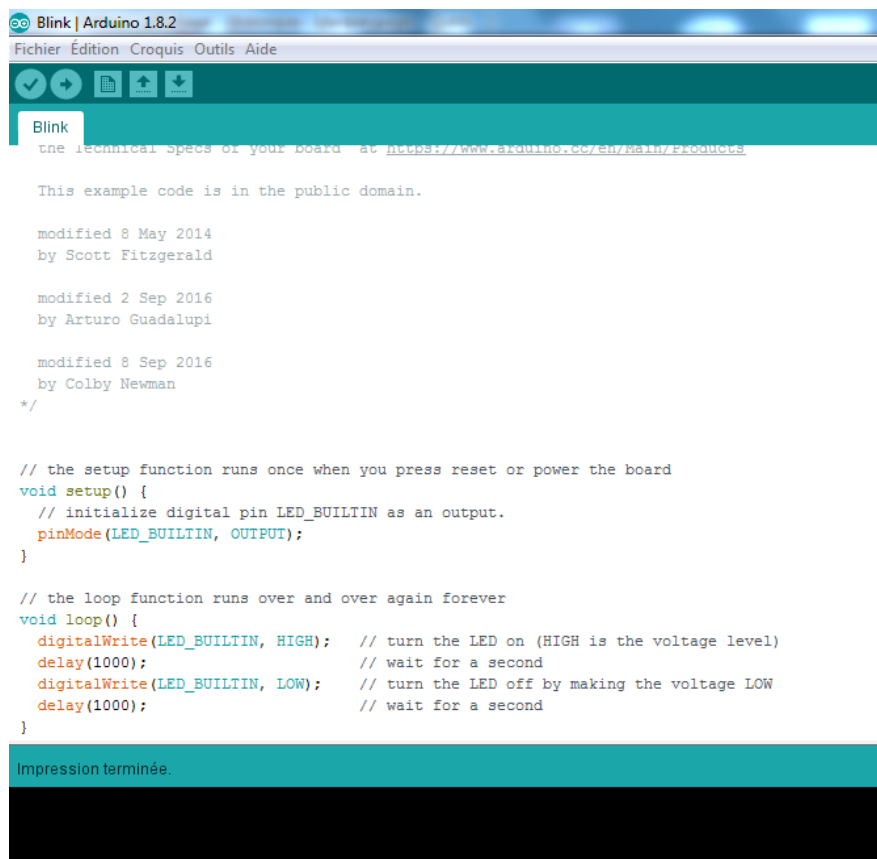
La fonction setup() est exécutée une seule fois lors du démarrage de la carte Arduino. On place dans le corps de cette fonction tout ce qui touche à l'initialisation du matériel (entrée, sortie, capteurs...).

La fonction loop() est appelée en boucle tant que la carte Arduino est alimentée. C'est dans le corps de cette fonction que l'on place le programme.

Il convient de consulter le site Arduino : <http://www.arduino.cc> et pour les éléments (fonctions, variables et structure) du langage de programmation <https://www.arduino.cc/reference/en/>. Traduit en français (partiellement) : <http://www.arduino.cc/fr>.

On se propose d'analyser quelques programmes sous Arduino cités en exemple par le constructeur. Ces programmes sont accessibles à partir de l'onglet « fichier »/ « exemples ». Il convient de consulter les schémas structurels de la carte « Arduino » (Documentations Techniques) lors de la connexion de composants externes : led, potentiomètre,

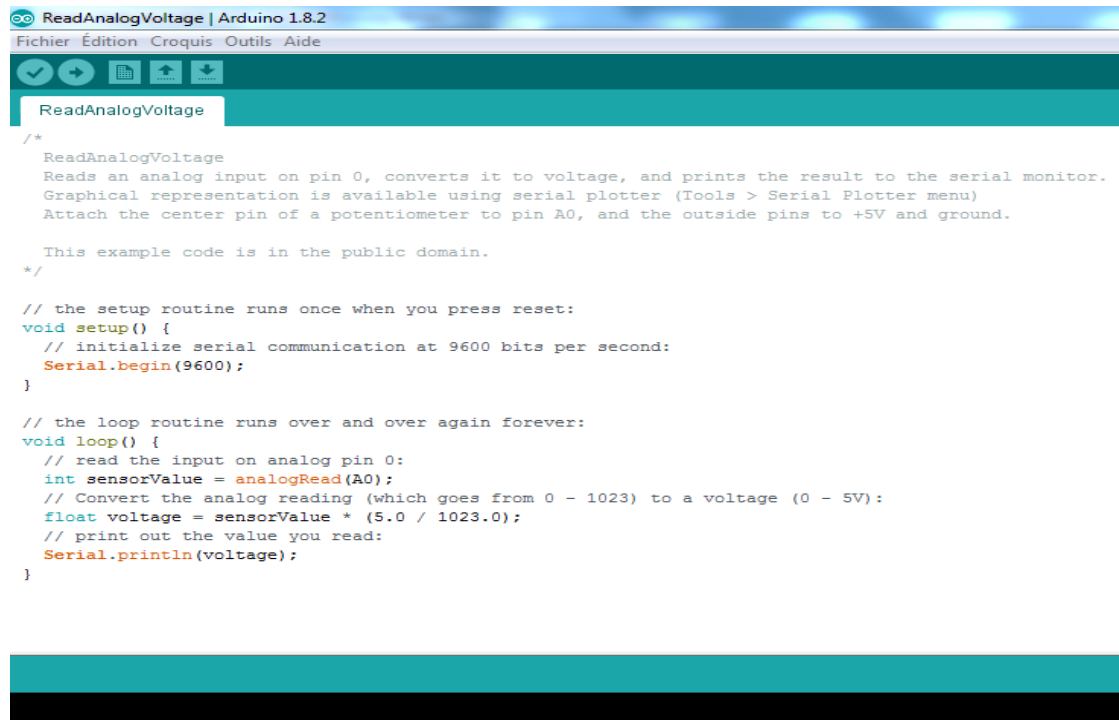
4.1 - Programme « Blink »



Ce programme permet de faire clignoter une Led branchée sur la broche D13 (Numéro de broche 13). La constante LED_BUILTIN = 13 est un mot clé du langage.

Compiler, téléverser et tester le programme. Modifier le programme afin de faire clignoter une Led branchée sur la broche 3. Il convient de consulter les éléments (fonctions, variables et structure) du langage de programmation <https://www.arduino.cc/reference/en/>

4.2 – Programme ReadAnalogVoltage

The image shows a screenshot of the Arduino IDE interface. The title bar reads "ReadAnalogVoltage | Arduino 1.8.2". The menu bar includes "Fichier", "Édition", "Croquis", "Outils", and "Aide". The toolbar contains icons for opening files, saving, and other standard IDE functions. The main text area displays the following code:

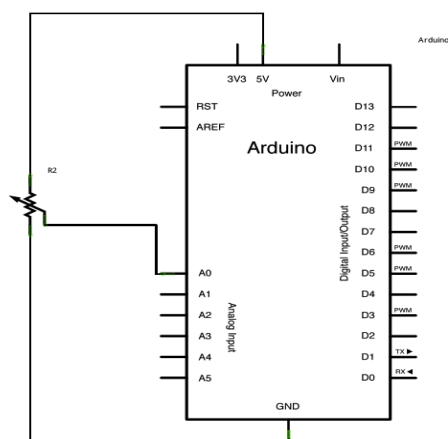
```
/*
  ReadAnalogVoltage
  Reads an analog input on pin 0, converts it to voltage, and prints the result to the serial monitor.
  Graphical representation is available using serial plotter (Tools > Serial Plotter menu)
  Attach the center pin of a potentiometer to pin A0, and the outside pins to +5V and ground.

  This example code is in the public domain.
*/

// the setup routine runs once when you press reset:
void setup() {
  // initialize serial communication at 9600 bits per second:
  Serial.begin(9600);
}

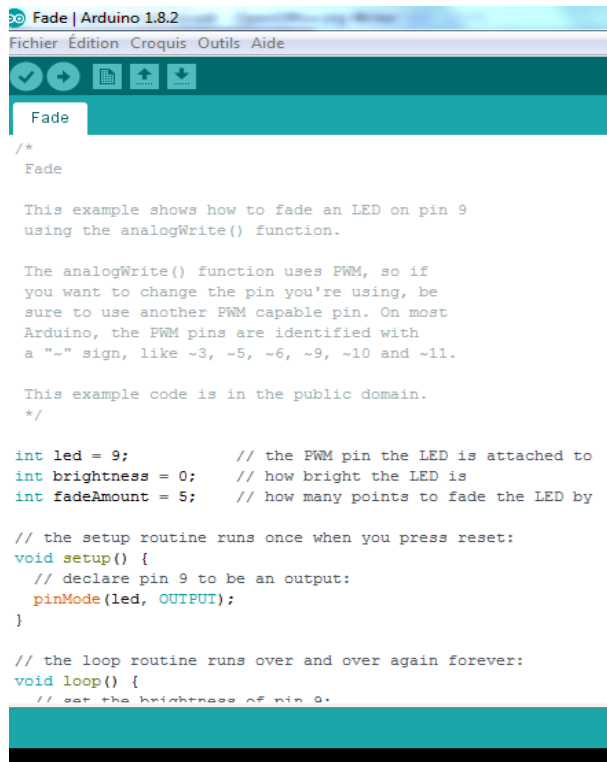
// the loop routine runs over and over again forever:
void loop() {
  // read the input on analog pin 0:
  int sensorValue = analogRead(A0);
  // Convert the analog reading (which goes from 0 - 1023) to a voltage (0 - 5V):
  float voltage = sensorValue * (5.0 / 1023.0);
  // print out the value you read:
  Serial.println(voltage);
}
```

Ce programme permet de lire une entrée Analogique A0 et d'afficher la valeur sur le terminal. Pour tester ce programme il convient de brancher un potentiomètre sur l'entrée A0.



Compiler, téléverser et tester le programme. Le potentiomètre a une valeur nominale de 100kΩ. Il convient de consulter les éléments (fonctions, variables et structure) du langage de programmation <https://www.arduino.cc/reference/en/>

4.3 – Programme Fade

A screenshot of the Arduino IDE interface. The title bar reads 'Fade | Arduino 1.8.2'. The menu bar includes 'Fichier', 'Édition', 'Croquis', 'Outils', and 'Aide'. Below the menu bar is a toolbar with icons for opening, saving, and other functions. The main text area displays the code for the 'Fade' program. The code includes comments explaining the use of the analogWrite() function and PWM pins. It defines variables for the LED pin (9), initial brightness (0), and fade amount (5). The setup() function configures pin 9 as an output. The loop() function continuously fades the LED by incrementing the brightness and wrapping it back to 0 when it reaches 255.

```
/*
  Fade

  This example shows how to fade an LED on pin 9
  using the analogWrite() function.

  The analogWrite() function uses PWM, so if
  you want to change the pin you're using, be
  sure to use another PWM capable pin. On most
  Arduino, the PWM pins are identified with
  a "~" sign, like ~3, ~5, ~6, ~9, ~10 and ~11.

  This example code is in the public domain.
  */

int led = 9;          // the PWM pin the LED is attached to
int brightness = 0;   // how bright the LED is
int fadeAmount = 5;   // how many points to fade the LED by

// the setup routine runs once when you press reset:
void setup() {
  // declare pin 9 to be an output:
  pinMode(led, OUTPUT);
}

// the loop routine runs over and over again forever:
void loop() {
  // set the brightness of pin 9:
  analogWrite(led, brightness);

  brightness = brightness + fadeAmount;

  // fade out from 0 to 255 in increments of fadeAmount = 5:
  if (brightness >= 255) {
    brightness = 0;
  }

  delay(30); // wait 30 milliseconds to turn the LED on
}
```

Ce programme permet de moduler l'éclairage d'une Led branchée sur la broche 9.

Compiler, téléverser et tester le programme. Il convient de consulter les éléments (fonctions, variables et structure) du langage de programmation <https://www.arduino.cc/reference/en/>

Le site Arduino : <http://www.arduino.cc>

Traduit en français (partiellement) : <http://www.arduino.cc/fr>